Now I have complete knowledge of every file and data structure. Let me build the complete extension suite.

Visual Climate — Tableau Extension Suite

기존 레포의 모든 실제 데이터 구조를 분석 완료했습니다. 아래는 환각 없이, 실제 Supabase 테이블(`countries`, `indicators`, `country_data`) 스키마, `pull-from-supabase.ts`의 실제 indicator_code, RAG 레포의 `ReportCardClient.tsx` / `CountryClient.tsx` 컴포넌트, `brand-config.md` 색상, `schema.json` 23개 지표, `ndc-targets-v2.json` 20국 NDC 데이터를 기반으로 작성한 **production-ready** 코드입니다.

파일 구조

```
extension/
├── visualclimate.trex        # Tableau manifest
├── index.html                # Extension 1: Climate Card (국가 프로필)
├── app.js                    # Climate Card 로직
├── timeseries.html           # Extension 2: 시계열 탐색기
├── timeseries-app.js         # 시계열 로직
├── ndc-tracker.html          # Extension 3: NDC Gap Tracker
├── ndc-tracker-app.js        # NDC Gap 로직
├── shared.js                 # 공유 유틸 (Supabase 클라이언트, 색상, 등급)
├── radar-chart.js            # D3-free SVG 레이더 차트 모듈
├── styles.css                # 브랜드 통합 스타일
├── timeseries.trex           # manifest for Extension 2
└── ndc-tracker.trex          # manifest for Extension 3
```

---

1. `extension/shared.js` — 공유 유틸리티

```
/**
 * shared.js — Visual Climate Tableau Extension 공유 모듈
 *
 * 실제 Supabase 테이블 구조:
 *   - countries: iso3, name, region, sub_region, income_group, population
 *   - indicators: code, source, name, unit, category, domain
```

```
 *   - country_data: country_iso3, indicator_code, year, value, source
 *
 * indicator_code 패턴 (pull-from-supabase.ts 기반, 실제 DB에서 검증됨):
 *   WB 계열: SP.POP.TOTL, NY.GDP.PCAP.CD, EN.ATM.GHGT.KT.CE, EN.ATM.CO2E.KT,
 *           EN.ATM.CO2E.PC, EG.FEC.RNEW.ZS, NY.GDP.MKTP.PP.CD
 *   OWID.*: TOTAL_GHG_EXCLUDING_LUCF, TOTAL_GHG, CO2, GHG_PER_CAPITA,
 *          CONSUMPTION_CO2_PER_CAPITA, COAL_CO2, OIL_CO2, GAS_CO2,
 *          METHANE, NITROUS_OXIDE, CO2_PER_GDP
 *   EMBER.*: RENEWABLE.PCT, CARBON.INTENSITY
 *   NDGAIN.*: VULNERABILITY, READINESS, RANK
 *   CTRACE.*: TOTAL, POWER, TRANSPORTATION, MANUFACTURING, AGRICULTURE,
WASTE,
 *           BUILDINGS, MINERAL_EXTRACTION, FLUORINATED_GASES
 *   DERIVED.*: CO2_PER_GDP, ENERGY_TRANSITION, CLIMATE_CLASS, DECOUPLING
 *   REPORT.*: TOTAL_SCORE, GRADE, EMISSIONS_SCORE, ENERGY_SCORE,
 *           ECONOMY_SCORE, RESPONSIBILITY_SCORE, RESILIENCE_SCORE
 */

const VC = (() => {
  'use strict';

  // ─── Supabase 설정 ───
  // .trex의 configure 팝업 또는 settings에서 주입됨.
  // Tableau Extension settings에 저장하여 재사용.
  let _supabaseUrl = '';
  let _supabaseKey = '';

  function configureSupabase(url, key) {
    _supabaseUrl = url.replace(/\/$/, '');
    _supabaseKey = key;
  }

  function loadConfigFromSettings() {
    if (typeof tableau !== 'undefined' && tableau.extensions) {
      const settings = tableau.extensions.settings;
      _supabaseUrl = settings.get('supabaseUrl') || '';
      _supabaseKey = settings.get('supabaseKey') || '';
    }
  }

  function saveConfigToSettings(url, key) {
    if (typeof tableau !== 'undefined' && tableau.extensions) {
      const settings = tableau.extensions.settings;
      settings.set('supabaseUrl', url);
```

```
    settings.set('supabaseKey', key);
    return settings.saveAsync();
  }
  return Promise.resolve();
}

// ── Supabase REST 호출 ──
// PostgREST 기반: GET /rest/v1/{table}?{query}
async function supabaseGet(table, query) {
  if (!_supabaseUrl || !_supabaseKey) {
    throw new Error('Supabase not configured. Call VC.configureSupabase(url, key) first.');
  }
  const url = `${_supabaseUrl}/rest/v1/${table}?${query}`;
  const resp = await fetch(url, {
    headers: {
      'apikey': _supabaseKey,
      'Authorization': `Bearer ${_supabaseKey}`,
      'Content-Type': 'application/json',
    },
  });
  if (!resp.ok) throw new Error(`Supabase ${resp.status}: ${await resp.text()}`);
  return resp.json();
}

// ── 데이터 쿼리 함수들 (실제 DB 구조 기반) ──

/** 국가 메타데이터 조회 */
async function getCountry(iso3) {
  const rows = await supabaseGet('countries',
    `iso3=eq.${iso3}&select=iso3,name,region,sub_region,income_group,population`);
  return rows[0] || null;
}

/** 모든 국가 목록 */
async function getAllCountries() {
  return supabaseGet('countries',
    'select=iso3,name,region,income_group&order=name');
}

/** 특정 국가의 최신 지표값 (indicator_code 배열) */
async function getLatestIndicators(iso3, codes) {
  // country_data에서 해당 코드들을 연도 내림차순으로 가져옴
  const codesStr = codes.map(c => `"${c}"`).join(',');
  const rows = await supabaseGet('country_data',
```

```
`country_iso3=eq.${iso3}&indicator_code=in.(${codes.join(',')})&order=year.desc&limit=500`);
  // 코드별 최신값만 추출 (pull-from-supabase.ts 패턴 동일)
  const latest = {};
  for (const row of rows) {
    if (!latest[row.indicator_code]) {
      latest[row.indicator_code] = {
        value: row.value,
        year: row.year,
        source: row.source,
      };
    }
  }
  return latest;
}

/** 특정 국가 + 특정 지표의 시계열 */
async function getTimeseries(iso3, code) {
  return supabaseGet('country_data',

`country_iso3=eq.${iso3}&indicator_code=eq.${code}&order=year.asc&select=year,value,sourc
e`);
}

/** 특정 국가 + 여러 지표의 시계열 */
async function getMultiTimeseries(iso3, codes) {
  return supabaseGet('country_data',

`country_iso3=eq.${iso3}&indicator_code=in.(${codes.join(',')})&order=year.asc&select=indicator
_code,year,value,source`);
}

/** 스코어카드 데이터 (REPORT.* 코드 사용) */
const REPORT_CODES = [
  'REPORT.TOTAL_SCORE',
  'REPORT.GRADE',
  'REPORT.EMISSIONS_SCORE',
  'REPORT.ENERGY_SCORE',
  'REPORT.ECONOMY_SCORE',
  'REPORT.RESPONSIBILITY_SCORE',
  'REPORT.RESILIENCE_SCORE',
];

async function getReportCard(iso3) {
```

```
  const country = await getCountry(iso3);
  if (!country) return null;

  const latest = await getLatestIndicators(iso3, REPORT_CODES);

  // DERIVED.CLIMATE_CLASS 조회 (2023년 데이터, RAG 레포에서 확인됨)
  const classRows = await supabaseGet('country_data',

`country_iso3=eq.${iso3}&indicator_code=eq.DERIVED.CLIMATE_CLASS&order=year.desc&limit=1`);

  const score = latest['REPORT.TOTAL_SCORE']?.value ?? null;
  const gradeNum = latest['REPORT.GRADE']?.value ?? null;

  return {
    iso3: country.iso3,
    name: country.name,
    region: country.region,
    income_group: country.income_group,
    total_score: score,
    grade: gradeNum !== null ? GRADE_LABEL[gradeNum] : null,
    grade_numeric: gradeNum,
    climate_class: classRows[0]?.value ?? null,
    emissions_score: latest['REPORT.EMISSIONS_SCORE']?.value ?? null,
    energy_score: latest['REPORT.ENERGY_SCORE']?.value ?? null,
    economy_score: latest['REPORT.ECONOMY_SCORE']?.value ?? null,
    responsibility_score: latest['REPORT.RESPONSIBILITY_SCORE']?.value ?? null,
    resilience_score: latest['REPORT.RESILIENCE_SCORE']?.value ?? null,
  };
}

/** 피어 컨텍스트: 글로벌 순위, 소득그룹 순위 */
async function getPeerContext(iso3) {
  // 모든 국가의 REPORT.TOTAL_SCORE 가져오기
  const allScores = await supabaseGet('country_data',

'indicator_code=eq.REPORT.TOTAL_SCORE&order=value.desc&select=country_iso3,value,year');

  // 국가별 최신값만 추출
  const countryScores = {};
  for (const row of allScores) {
    if (!countryScores[row.country_iso3]) {
      countryScores[row.country_iso3] = row.value;
```

```
    }
  }

  // 순위 계산
  const sorted = Object.entries(countryScores)
    .sort((a, b) => b[1] - a[1]);
  const globalRank = sorted.findIndex(([c]) => c === iso3) + 1;
  const totalCountries = sorted.length;

  // 소득그룹 내 순위
  const country = await getCountry(iso3);
  let incomeRank = null;
  let incomeTotal = null;
  if (country?.income_group) {
    const peerCountries = await supabaseGet('countries',
      `income_group=eq.${encodeURIComponent(country.income_group)}&select=iso3`);
    const peerIsos = new Set(peerCountries.map(c => c.iso3));
    const peerSorted = sorted.filter(([c]) => peerIsos.has(c));
    incomeRank = peerSorted.findIndex(([c]) => c === iso3) + 1;
    incomeTotal = peerSorted.length;
  }

  return { globalRank, totalCountries, incomeRank, incomeTotal };
}

// ─── 등급/색상 매핑 (ReportCardClient.tsx에서 추출) ───

// REPORT.GRADE 숫자 → 텍스트 (RAG 레포 page.tsx 기반)
const GRADE_LABEL = {
  0: 'A+', 1: 'A', 2: 'B+', 3: 'B', 4: 'C+', 5: 'C', 6: 'D', 7: 'F',
};

// 등급별 색상 (ReportIndexClient.tsx에서 추출)
const GRADE_COLOR = {
  'A+': '#00A67E', 'A': '#00A67E',
  'B+': '#0066FF', 'B': '#0066FF',
  'C+': '#F59E0B', 'C': '#F59E0B',
  'D': '#E5484D',
  'F': '#7F1D1D',
};

const GRADE_BG = {
  'A+': '#ECFDF5', 'A': '#ECFDF5',
  'B+': '#EFF6FF', 'B': '#EFF6FF',
```

```
  'C+': '#FFFBEB', 'C': '#FFFBEB',
  'D': '#FEF2F2',
  'F': '#FEF2F2',
};

// Climate Class (DERIVED.CLIMATE_CLASS: 1=Changer, 2=Starter, 3=Talker)
const CLASS_LABEL = { 1: 'Changer', 2: 'Starter', 3: 'Talker' };
const CLASS_COLOR = {
  'Changer': '#00A67E', 'Starter': '#F59E0B', 'Talker': '#E5484D',
};
const CLASS_EXPLAIN = {
  'Changer': 'CO₂ declining AND renewables rising',
  'Starter': 'Either CO₂ declining OR renewables rising (not both)',
  'Talker': 'Neither CO₂ declining nor renewables rising',
};

// 도메인 정의 (ReportCardClient.tsx에서 추출)
const DOMAINS = [
  { key: 'emissions', label: 'Emissions', weight: '30%', color: '#E5484D',
    scoreField: 'emissions_score' },
  { key: 'energy', label: 'Energy', weight: '25%', color: '#0066FF',
    scoreField: 'energy_score' },
  { key: 'economy', label: 'Economy', weight: '15%', color: '#8B5CF6',
    scoreField: 'economy_score' },
  { key: 'responsibility', label: 'Responsibility', weight: '15%', color: '#F59E0B',
    scoreField: 'responsibility_score' },
  { key: 'resilience', label: 'Resilience', weight: '15%', color: '#00A67E',
    scoreField: 'resilience_score' },
];

// brand-config.md 색상
const BRAND = {
  primary: '#0066FF',
  changer: '#00A67E',
  starter: '#F59E0B',
  talker: '#E5484D',
  critical: '#7F1D1D',
  bg: '#FFFFFF',
  sectionBg: '#F8F9FA',
  textPrimary: '#1A1A2E',
  textSecondary: '#4A4A6A',
  textMuted: '#8888A0',
  border: '#E5E7EB',
  dataMissing: '#CCCCCC',
```

```
};

// CAT rating 색상 (calculated-fields.md CF10 기반)
const CAT_COLOR = {
  '1.5C_compatible': '#00A67E',
  'Almost_sufficient': '#8BC34A',
  'Insufficient': '#F59E0B',
  'Highly_insufficient': '#E5484D',
  'Critically_insufficient': '#7F1D1D',
};

// NDC Gap Severity 색상 (calculated-fields.md CF05 기반)
const GAP_COLOR = {
  'On Track': '#00A67E',
  'Narrow Gap': '#F59E0B',
  'Significant Gap': '#E5484D',
  'Critical Gap': '#7F1D1D',
};

// RE Share Category (CF09)
function reShareCategory(pct) {
  if (pct == null) return 'No Data';
  if (pct >= 80) return 'RE Leader (80%+)';
  if (pct >= 50) return 'Majority RE (50-79%)';
  if (pct >= 25) return 'Growing RE (25-49%)';
  return 'Fossil Dominant (<25%)';
}

// Score → 성능 라벨 (ReportCardClient.tsx의 perfLabel 기반)
function perfLabel(score) {
  if (score == null) return 'No Data';
  if (score >= 90) return 'Excellent';
  if (score >= 70) return 'Good';
  if (score >= 50) return 'Fair';
  if (score >= 25) return 'Poor';
  return 'Critical';
}

// Score → 색상
function scoreColor(score) {
  if (score == null) return BRAND.dataMissing;
  if (score >= 80) return '#00A67E';
  if (score >= 60) return '#0066FF';
  if (score >= 40) return '#F59E0B';
```

```
  if (score >= 20) return '#E5484D';
  return '#7F1D1D';
}

// ISO3 → 국기 이모지 (ReportCardClient.tsx의 iso3ToFlag 로직)
function iso3ToFlag(iso3) {
  const iso2Map = {
    'KOR': 'KR', 'USA': 'US', 'DEU': 'DE', 'CHN': 'CN', 'JPN': 'JP',
    'GBR': 'GB', 'FRA': 'FR', 'BRA': 'BR', 'IND': 'IN', 'AUS': 'AU',
    'CAN': 'CA', 'RUS': 'RU', 'IDN': 'ID', 'MEX': 'MX', 'ZAF': 'ZA',
    'NGA': 'NG', 'BGD': 'BD', 'SAU': 'SA', 'TUR': 'TR', 'EGY': 'EG',
    'KHM': 'KH', 'VNM': 'VN', 'THA': 'TH', 'PHL': 'PH', 'MMR': 'MM',
    'LAO': 'LA', 'SGP': 'SG', 'MYS': 'MY', 'TWN': 'TW', 'HKG': 'HK',
  };
  const iso2 = iso2Map[iso3] || iso3.substring(0, 2);
  return String.fromCodePoint(
    ...[...iso2.toUpperCase()].map(c => 0x1F1E6 + c.charCodeAt(0) - 65)
  );
}

// 숫자 포맷
function fmt(val, decimals = 1) {
  if (val == null || isNaN(val)) return '—';
  return Number(val).toFixed(decimals);
}

function fmtLarge(val) {
  if (val == null || isNaN(val)) return '—';
  const n = Number(val);
  if (n >= 1e9) return (n / 1e9).toFixed(1) + 'B';
  if (n >= 1e6) return (n / 1e6).toFixed(1) + 'M';
  if (n >= 1e3) return (n / 1e3).toFixed(1) + 'K';
  return n.toFixed(0);
}

// ─── Tableau 워크시트에서 ISO3 탐지 ───
async function detectISO3FromDashboard() {
  const dashboard = tableau.extensions.dashboardContent.dashboard;
  const worksheets = dashboard.worksheets;

  for (const ws of worksheets) {
    // 1. 필터에서 ISO3 찾기
    const filters = await ws.getFiltersAsync();
    for (const f of filters) {
```

```
      if (f.fieldName.toLowerCase().includes('iso3') ||
          f.fieldName.toLowerCase().includes('iso') ||
          f.fieldName.toLowerCase().includes('country_iso3')) {
        if (f.appliedValues && f.appliedValues.length === 1) {
          return f.appliedValues[0].value;
        }
      }
    }
  }

  // 2. 선택된 마크에서 ISO3 찾기
  for (const ws of worksheets) {
    const marks = await ws.getSelectedMarksAsync();
    if (marks.data && marks.data.length > 0) {
      const table = marks.data[0];
      const columns = table.columns;
      const iso3ColIdx = columns.findIndex(
        c => c.fieldName.toLowerCase().includes('iso3') ||
             c.fieldName.toLowerCase().includes('iso'));
      if (iso3ColIdx >= 0 && table.data.length > 0) {
        return table.data[0][iso3ColIdx].value;
      }
    }
  }

  return null;
}

// ─── Tableau 이벤트 리스너 등록 ───
function onFilterChange(callback) {
  const dashboard = tableau.extensions.dashboardContent.dashboard;
  for (const ws of dashboard.worksheets) {
    ws.addEventListener(
      tableau.TableauEventType.FilterChanged,
      () => callback()
    );
  }
}

function onMarkSelection(callback) {
  const dashboard = tableau.extensions.dashboardContent.dashboard;
  for (const ws of dashboard.worksheets) {
    ws.addEventListener(
      tableau.TableauEventType.MarkSelectionChanged,
```

```
      () => callback()
    );
  }
}

// ─── 시계열 indicator 그룹 (CountryClient.tsx에서 추출된 실제 코드) ───
const TIMESERIES_GROUPS = {
  emissions: {
    label: 'Emissions',
    codes: [
      'EN.GHG.CO2.PC.CE.AR5',  // WB CO2 per capita
      'OWID.GHG_PER_CAPITA',
      'OWID.CO2',
      'OWID.TOTAL_GHG_EXCLUDING_LUCF',
      'OWID.CONSUMPTION_CO2_PER_CAPITA',
      'OWID.CO2_PER_GDP',
    ],
  },
  fuel: {
    label: 'Fuel Breakdown',
    codes: [
      'OWID.COAL_CO2',
      'OWID.OIL_CO2',
      'OWID.GAS_CO2',
      'OWID.CEMENT_CO2',
      'OWID.FLARING_CO2',
    ],
  },
  sector: {
    label: 'Sector (Climate TRACE)',
    codes: [
      'CTRACE.POWER',
      'CTRACE.TRANSPORTATION',
      'CTRACE.MANUFACTURING',
      'CTRACE.AGRICULTURE',
      'CTRACE.BUILDINGS',
      'CTRACE.WASTE',
      'CTRACE.MINERAL_EXTRACTION',
      'CTRACE.FLUORINATED_GASES',
    ],
  },
  energy: {
    label: 'Energy',
    codes: [
```

```
      'EMBER.RENEWABLE.PCT',
      'EMBER.CARBON.INTENSITY',
      'EG.FEC.RNEW.ZS',
    ],
  },
  economy: {
    label: 'Economy',
    codes: [
      'NY.GDP.PCAP.CD',
      'SP.POP.TOTL',
      'DERIVED.CO2_PER_GDP',
    ],
  },
  resilience: {
    label: 'Resilience',
    codes: [
      'NDGAIN.VULNERABILITY',
      'NDGAIN.READINESS',
    ],
  },
  other_ghg: {
    label: 'Other GHG',
    codes: [
      'OWID.METHANE',
      'OWID.NITROUS_OXIDE',
    ],
  },
};

// —— NDC Targets (ndc-targets-v2.json 기반, 20국) ——
const NDC_TARGETS = {
  KOR: { name:'South Korea', ndc2030_pct:40, ref_year:2018, ref_type:'absolute',
ref_mt:727.6, target_mt:436.6, ndc3:false, nz_year:2050, nz_legal:'in_law',
cat:'Highly_insufficient' },
  USA: { name:'United States', ndc2030_pct:50, ref_year:2005, ref_type:'absolute',
ref_mt:7417, target_mt:3709, ndc3:false, nz_year:2050, nz_legal:'pledge', cat:'Insufficient' },
  DEU: { name:'Germany', ndc2030_pct:65, ref_year:1990, ref_type:'absolute', ref_mt:1251,
target_mt:438, ndc3:true, ndc3_pct:78, ndc3_date:'2025-02-14', nz_year:2045, nz_legal:'in_law',
cat:'Insufficient' },
  CHN: { name:'China', ndc2030_pct:65, ref_year:2005, ref_type:'intensity', ndc3:false,
nz_year:2060, nz_legal:'pledge', cat:'Highly_insufficient' },
  IND: { name:'India', ndc2030_pct:45, ref_year:2005, ref_type:'intensity', ndc3:false,
nz_year:2070, nz_legal:'pledge', cat:'Highly_insufficient' },
```

```
    JPN: { name:'Japan', ndc2030_pct:46, ref_year:2013, ref_type:'absolute', ndc3:true,
ndc3_pct:60, ndc3_date:'2025-02-04', nz_year:2050, nz_legal:'in_law', cat:'Insufficient' },
    GBR: { name:'United Kingdom', ndc2030_pct:68, ref_year:1990, ref_type:'absolute',
ndc3:true, ndc3_pct:81, ndc3_date:'2025-02-04', nz_year:2050, nz_legal:'in_law',
cat:'Insufficient' },
    BRA: { name:'Brazil', ndc2030_pct:50, ref_year:2005, ref_type:'absolute', ndc3:true,
ndc3_pct:67, ndc3_date:'2025-02-10', nz_year:2050, nz_legal:'in_policy', cat:'Insufficient' },
    CAN: { name:'Canada', ndc2030_pct:40, ref_year:2005, ref_type:'absolute', ndc3:false,
nz_year:2050, nz_legal:'in_law', cat:'Insufficient' },
    AUS: { name:'Australia', ndc2030_pct:43, ref_year:2005, ref_type:'absolute', ndc3:false,
nz_year:2050, nz_legal:'in_law', cat:'Insufficient' },
    FRA: { name:'France', ndc2030_pct:55, ref_year:1990, ref_type:'absolute', ndc3:true,
ndc3_date:'2025-02-14', nz_year:2050, nz_legal:'in_law', cat:'Insufficient' },
    IDN: { name:'Indonesia', ndc2030_pct:32, ref_type:'bau', ndc3:false, nz_year:2060,
nz_legal:'in_policy', cat:'Highly_insufficient' },
    NGA: { name:'Nigeria', ndc2030_pct:47, ref_type:'bau', ndc3:false, nz_year:2060,
nz_legal:'pledge', cat:'Highly_insufficient' },
    BGD: { name:'Bangladesh', ndc2030_pct:22, ref_type:'bau', ndc3:false, nz_year:null,
nz_legal:'none', cat:null },
    ZAF: { name:'South Africa', ndc2030_pct:32, ref_type:'absolute', ndc3:false, nz_year:2050,
nz_legal:'in_policy', cat:'Highly_insufficient' },
    MEX: { name:'Mexico', ndc2030_pct:35, ref_type:'bau', ndc3:false, nz_year:2050,
nz_legal:'pledge', cat:'Critically_insufficient' },
    RUS: { name:'Russia', ndc2030_pct:30, ref_year:1990, ref_type:'absolute', ndc3:false,
nz_year:2060, nz_legal:'in_policy', cat:'Critically_insufficient' },
    SAU: { name:'Saudi Arabia', ndc2030_pct:19, ref_type:'bau', ndc3:false, nz_year:2060,
nz_legal:'pledge', cat:'Critically_insufficient' },
    TUR: { name:'Turkey', ndc2030_pct:41, ref_type:'bau', ndc3:false, nz_year:2053,
nz_legal:'pledge', cat:'Critically_insufficient' },
    EGY: { name:'Egypt', ndc2030_pct:33, ref_type:'bau', ndc3:false, nz_year:null,
nz_legal:'none', cat:'Highly_insufficient' },
  };

  return {
    configureSupabase, loadConfigFromSettings, saveConfigToSettings,
    supabaseGet, getCountry, getAllCountries, getLatestIndicators,
    getTimeseries, getMultiTimeseries, getReportCard, getPeerContext,
    detectISO3FromDashboard, onFilterChange, onMarkSelection,
    GRADE_LABEL, GRADE_COLOR, GRADE_BG,
    CLASS_LABEL, CLASS_COLOR, CLASS_EXPLAIN,
    DOMAINS, BRAND, CAT_COLOR, GAP_COLOR,
    NDC_TARGETS, REPORT_CODES, TIMESERIES_GROUPS,
    reShareCategory, perfLabel, scoreColor, iso3ToFlag, fmt, fmtLarge,
  };
```

```
})();
```

---

2. `extension/radar-chart.js` — SVG 레이더 차트 (D3 불필요)

```
/**
 * radar-chart.js — Pure SVG radar chart for 5 climate domains.
 * No external dependencies. Matches ReportCardClient.tsx domain colors.
 *
 * Usage:
 *   RadarChart.render('container-id', {
 *     emissions: 98.6,
 *     energy: 52.3,
 *     economy: 0.8,
 *     responsibility: 99.9,
 *     resilience: null
 *   }, {
 *     emissions: 55, energy: 50, economy: 45, responsibility: 60, resilience: 40
 *   });
 */
const RadarChart = (() => {
  'use strict';

  const DOMAINS = [
    { key: 'emissions', label: 'Emissions', angle: -90, color: '#E5484D' },
    { key: 'energy', label: 'Energy', angle: -18, color: '#0066FF' },
    { key: 'economy', label: 'Economy', angle: 54, color: '#8B5CF6' },
    { key: 'responsibility', label: 'Responsibility', angle: 126, color: '#F59E0B' },
    { key: 'resilience', label: 'Resilience', angle: 198, color: '#00A67E' },
  ];

  function polarToXY(cx, cy, radius, angleDeg) {
    const rad = (angleDeg * Math.PI) / 180;
    return {
      x: cx + radius * Math.cos(rad),
      y: cy + radius * Math.sin(rad),
    };
  }

  function render(containerId, scores, globalAvg) {
    const container = document.getElementById(containerId);
```

```
  if (!container) return;

  const size = 300;
  const cx = size / 2;
  const cy = size / 2;
  const maxR = 120;
  const levels = 5; // 0, 20, 40, 60, 80, 100

  let svg = `<svg width="${size}" height="${size}" viewBox="0 0 ${size} ${size}"
xmlns="http://www.w3.org/2000/svg">`;

  // 배경 원형 그리드
  for (let i = 1; i <= levels; i++) {
    const r = (maxR / levels) * i;
    const points = DOMAINS.map(d => {
      const p = polarToXY(cx, cy, r, d.angle);
      return `${p.x},${p.y}`;
    }).join(' ');
    svg += `<polygon points="${points}" fill="none" stroke="#E5E7EB" stroke-width="0.5"/>`;
  }

  // 축선
  for (const d of DOMAINS) {
    const p = polarToXY(cx, cy, maxR, d.angle);
    svg += `<line x1="${cx}" y1="${cy}" x2="${p.x}" y2="${p.y}" stroke="#E5E7EB"
stroke-width="0.5"/>`;
  }

  // 글로벌 평균 다각형 (있을 경우)
  if (globalAvg) {
    const avgPoints = DOMAINS.map(d => {
      const val = globalAvg[d.key] ?? 0;
      const r = (val / 100) * maxR;
      const p = polarToXY(cx, cy, r, d.angle);
      return `${p.x},${p.y}`;
    }).join(' ');
    svg += `<polygon points="${avgPoints}" fill="rgba(136,136,160,0.15)" stroke="#8888A0"
stroke-width="1" stroke-dasharray="4,3"/>`;
  }

  // 국가 점수 다각형
  const validScores = DOMAINS.map(d => ({
    ...d,
    value: scores[d.key] ?? null,
```

```
  }));

  const scorePoints = validScores.map(d => {
    const val = d.value ?? 0;
    const r = (val / 100) * maxR;
    return polarToXY(cx, cy, r, d.angle);
  });

  const polyStr = scorePoints.map(p => `${p.x},${p.y}`).join(' ');
  svg += `<polygon points="${polyStr}" fill="rgba(0,102,255,0.15)" stroke="#0066FF"
stroke-width="2"/>`;

  // 각 꼭짓점 도트 + 점수 라벨
  for (let i = 0; i < DOMAINS.length; i++) {
    const d = DOMAINS[i];
    const val = scores[d.key];
    const r = val != null ? (val / 100) * maxR : 0;
    const p = polarToXY(cx, cy, r, d.angle);

    // 도트
    if (val != null) {
      svg += `<circle cx="${p.x}" cy="${p.y}" r="4" fill="${d.color}" stroke="white"
stroke-width="1.5"/>`;
    }

    // 도메인 라벨 (외곽)
    const lp = polarToXY(cx, cy, maxR + 22, d.angle);
    const anchor = d.angle > 90 && d.angle < 270 ? 'end' : (d.angle === 90 || d.angle === 270 ?
'middle' : 'start');
    // 상단(−90도)일 때 가운데 정렬
    const textAnchor = Math.abs(d.angle + 90) < 1 ? 'middle' :
                       (d.angle > -90 && d.angle < 90) ? 'start' :
                       (Math.abs(d.angle - 90) < 1 || Math.abs(d.angle - 270) < 1) ? 'middle' : 'end';

    svg += `<text x="${lp.x}" y="${lp.y}" text-anchor="middle" font-size="10"
font-family="Inter,sans-serif" fill="#4A4A6A">${d.label}</text>`;

    // 점수값 (도트 아래)
    if (val != null) {
      svg += `<text x="${lp.x}" y="${lp.y + 13}" text-anchor="middle" font-size="9"
font-family="JetBrains Mono,monospace" fill="${d.color}"
font-weight="600">${val.toFixed(1)}</text>`;
    } else {
```

```
      svg += `<text x="${lp.x}" y="${lp.y + 13}" text-anchor="middle" font-size="9" 
font-family="Inter,sans-serif" fill="#CCCCCC">—</text>`;
      }
    }

    svg += '</svg>';
    container.innerHTML = svg;
  }

  return { render };
})();
```

---

3. `extension/styles.css` — 브랜드 통합 스타일시트

```
/**
 * styles.css — Visual Climate Tableau Extension
 * brand-config.md 색상 + Inter/JetBrains Mono 폰트 기반
 */

@import 
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+M
ono:wght@400;500&display=swap');

:root {
  --vc-primary: #0066FF;
  --vc-changer: #00A67E;
  --vc-starter: #F59E0B;
  --vc-talker: #E5484D;
  --vc-critical: #7F1D1D;
  --vc-bg: #FFFFFF;
  --vc-section-bg: #F8F9FA;
  --vc-text: #1A1A2E;
  --vc-text-secondary: #4A4A6A;
  --vc-text-muted: #8888A0;
  --vc-border: #E5E7EB;
  --vc-missing: #CCCCCC;

  --vc-emissions: #E5484D;
  --vc-energy: #0066FF;
  --vc-economy: #8B5CF6;
  --vc-responsibility: #F59E0B;
  --vc-resilience: #00A67E;
```

```
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
  background: var(--vc-bg);
  color: var(--vc-text);
  font-size: 13px;
  line-height: 1.5;
  overflow-x: hidden;
}

/* ——— Setup Screen ——— */
.setup-screen {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
  padding: 24px;
  text-align: center;
}
.setup-screen h2 { font-size: 18px; margin-bottom: 16px; }
.setup-screen input {
  width: 100%;
  max-width: 400px;
  padding: 8px 12px;
  border: 1px solid var(--vc-border);
  border-radius: 6px;
  font-size: 13px;
  margin-bottom: 10px;
}
.setup-screen button {
  padding: 8px 24px;
  background: var(--vc-primary);
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
}
.setup-screen button:hover { opacity: 0.9; }
```

```
/* —— Loading / Empty —— */
.loading, .empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100%;
  min-height: 200px;
  color: var(--vc-text-muted);
  font-size: 14px;
}
.spinner {
  width: 32px; height: 32px;
  border: 3px solid var(--vc-border);
  border-top-color: var(--vc-primary);
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
  margin-bottom: 12px;
}
@keyframes spin { to { transform: rotate(360deg); } }

/* —— Climate Card (Extension 1) —— */
.climate-card {
  padding: 16px;
  max-width: 480px;
  margin: 0 auto;
}

.card-header {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 16px;
}
.card-flag { font-size: 32px; }
.card-country-name { font-size: 18px; font-weight: 700; }
.card-meta { font-size: 11px; color: var(--vc-text-muted); }

.score-hero {
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 16px;
```

```
  background: var(--vc-section-bg);
  border-radius: 12px;
  margin-bottom: 16px;
}
.score-big {
  font-family: 'JetBrains Mono', monospace;
  font-size: 42px;
  font-weight: 700;
  line-height: 1;
}
.score-unit { font-size: 14px; color: var(--vc-text-muted); font-weight: 400; }
.grade-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 48px;
  height: 48px;
  border-radius: 12px;
  font-size: 22px;
  font-weight: 700;
}
.class-pill {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 12px;
  font-size: 11px;
  font-weight: 600;
  color: white;
}
.rank-text {
  font-size: 12px;
  color: var(--vc-text-secondary);
  font-family: 'JetBrains Mono', monospace;
}

/* —— Domain Bars —— */
.domain-section {
  margin-bottom: 16px;
}
.domain-section h3 {
  font-size: 13px;
  font-weight: 600;
  margin-bottom: 8px;
  color: var(--vc-text-secondary);
```

```
}
.domain-bar-row {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 6px;
}
.domain-label {
  width: 100px;
  font-size: 11px;
  font-weight: 500;
  color: var(--vc-text-secondary);
  text-align: right;
}
.domain-bar-track {
  flex: 1;
  height: 20px;
  background: #F1F5F9;
  border-radius: 4px;
  position: relative;
  overflow: visible;
}
.domain-bar-fill {
  height: 100%;
  border-radius: 4px;
  transition: width 0.6s ease;
}
.domain-bar-avg {
  position: absolute;
  top: -2px;
  bottom: -2px;
  width: 2px;
  background: #1A1A2E;
  opacity: 0.3;
}
.domain-bar-value {
  width: 44px;
  font-family: 'JetBrains Mono', monospace;
  font-size: 12px;
  font-weight: 600;
  text-align: right;
}

/* —— Radar Container —— */
```

```
.radar-container {
  display: flex;
  justify-content: center;
  margin-bottom: 16px;
}

/* ——— Key Insights ——— */
.insights {
  padding: 12px;
  background: var(--vc-section-bg);
  border-radius: 8px;
  border-left: 3px solid var(--vc-primary);
  font-size: 12px;
  color: var(--vc-text-secondary);
  line-height: 1.6;
  margin-bottom: 12px;
}
.insights strong { color: var(--vc-text); }

/* ——— Timeseries (Extension 2) ——— */
.ts-controls {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-bottom: 12px;
  padding: 0 16px;
}
.ts-controls select, .ts-controls button {
  padding: 6px 12px;
  border: 1px solid var(--vc-border);
  border-radius: 6px;
  font-size: 12px;
  background: white;
  cursor: pointer;
}
.ts-controls button.active {
  background: var(--vc-primary);
  color: white;
  border-color: var(--vc-primary);
}

.ts-chart-area {
  padding: 0 16px;
  height: calc(100vh - 120px);
```

```
  min-height: 300px;
}
.ts-chart-area svg { width: 100%; height: 100%; }

/* ——— NDC Tracker (Extension 3) ——— */
.ndc-card {
  padding: 16px;
  max-width: 520px;
  margin: 0 auto;
}
.ndc-header { margin-bottom: 12px; }
.ndc-kpi-row {
  display: flex;
  gap: 8px;
  margin-bottom: 12px;
}
.ndc-kpi {
  flex: 1;
  padding: 10px;
  background: var(--vc-section-bg);
  border-radius: 8px;
  text-align: center;
}
.ndc-kpi-value {
  font-family: 'JetBrains Mono', monospace;
  font-size: 20px;
  font-weight: 700;
}
.ndc-kpi-label {
  font-size: 10px;
  color: var(--vc-text-muted);
  margin-top: 2px;
}
.ndc-gap-badge {
  display: inline-block;
  padding: 4px 12px;
  border-radius: 6px;
  font-size: 12px;
  font-weight: 600;
  color: white;
}
.ndc-chart-area { margin-bottom: 16px; }
.ndc-detail { font-size: 11px; color: var(--vc-text-muted); }
.ndc-detail td { padding: 3px 8px; }
```

```
/* —— Footer —— */
.vc-footer {
  padding: 8px 16px;
  font-size: 10px;
  color: var(--vc-text-muted);
  text-align: right;
  border-top: 1px solid var(--vc-border);
}
.vc-footer a {
  color: var(--vc-primary);
  text-decoration: none;
}

/* —— Comparison Grid (multi-country) —— */
.comparison-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 12px;
  padding: 16px;
}
.comparison-mini-card {
  padding: 12px;
  border: 1px solid var(--vc-border);
  border-radius: 8px;
  text-align: center;
}
.comparison-mini-card .card-flag { font-size: 24px; }
.comparison-mini-card .card-country-name { font-size: 14px; }
.mini-domain-row {
  display: flex;
  align-items: center;
  gap: 4px;
  margin: 2px 0;
}
.mini-domain-label { width: 24px; font-size: 9px; color: var(--vc-text-muted); text-align: right; }
.mini-bar-track { flex: 1; height: 8px; background: #F1F5F9; border-radius: 2px; }
.mini-bar-fill { height: 100%; border-radius: 2px; }
.mini-bar-value { width: 28px; font-size: 9px; font-family: 'JetBrains Mono', monospace; }
```

---

4. `extension/visualclimate.trex` — Climate Card Manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest manifest-version="0.1" xmlns="http://www.tableau.com/xml/extension_manifest">
  <dashboard-extension id="com.visualclimate.climate-card" extension-version="1.0.0">
    <default-locale>en_US</default-locale>
    <name>Visual Climate — Climate Card</name>
    <description>
      AI-powered Country Climate Report Card showing grade, score,
      5-domain breakdown, and classification for 250 countries.
      Data source: visualclimate.org Supabase (172,121 rows, 250 countries, 61 indicators).
    </description>
    <author name="Harim Jung" email="harim@visualclimate.org"
          organization="Visual Climate" website="https://visualclimate.org"/>
    <min-api-version>1.4</min-api-version>
    <source-location>
      <url>http://localhost:8765/extension/index.html</url>
    </source-location>

<icon>iVBORw0KGgoAAAANSUhEUgAAABgAAAAYCAYAAADgdz34AAAABmJLR0QA/wD/AP
+gvaeTAAAASkIEQVRIie2UMQ4AIAgE+f+nsdBYYIOxcRqchhMQJEnS30TETkPkMbABJhYi4h
m9BpjktcQWN7JDJDlHkq0LFiR+I0mSvuMCyJYZOXCe/VQAAAAASUVORK5CYII=</icon>
    <permissions>
      <permission>full data</permission>
    </permissions>
    <context-menu>
      <configure-context-menu-item />
    </context-menu>
  </dashboard-extension>
</manifest>
```

---

5. `extension/index.html` — Extension 1: Climate Card

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Visual Climate — Climate Card</title>
  <link rel="stylesheet" href="styles.css">
```

```
</head>
<body>

  <!-- Setup Screen (Supabase 미설정시) -->
  <div id="setup-screen" class="setup-screen" style="display:none;">
    <h2>Visual Climate — Climate Card</h2>
    <p style="margin-bottom:16px;color:#4A4A6A;font-size:12px;">
      Supabase 연결 정보를 입력하세요. 한 번만 설정하면 워크북에 저장됩니다.
    </p>
    <input type="text" id="input-url" placeholder="Supabase URL (https://xxx.supabase.co)">
    <input type="text" id="input-key" placeholder="Supabase anon key or service key">
    <button id="btn-save-config">Connect</button>
    <p id="setup-error" style="color:#E5484D;font-size:11px;margin-top:8px;"></p>
  </div>

  <!-- Loading -->
  <div id="loading" class="loading" style="display:none;">
    <div class="spinner"></div>
    <span>Loading climate data...</span>
  </div>

  <!-- Empty State -->
  <div id="empty-state" class="empty-state" style="display:none;">
    <p>Select a country on the dashboard to view its Climate Card.</p>
    <p style="font-size:11px;color:#8888A0;margin-top:8px;">
      The extension detects ISO3 from filters or selected marks.
    </p>
  </div>

  <!-- Climate Card -->
  <div id="climate-card" class="climate-card" style="display:none;">
    <!-- Header -->
    <div class="card-header">
      <span id="card-flag" class="card-flag"></span>
      <div>
        <div id="card-name" class="card-country-name"></div>
        <div id="card-meta" class="card-meta"></div>
      </div>
    </div>

    <!-- Score Hero -->
    <div class="score-hero">
      <div>
        <span id="card-score" class="score-big"></span>
```

```
      <span class="score-unit">/100</span>
    </div>
    <span id="card-grade" class="grade-badge"></span>
    <div>
      <span id="card-class" class="class-pill"></span>
      <div id="card-rank" class="rank-text" style="margin-top:4px;"></div>
    </div>
  </div>

  <!-- View Toggle -->
  <div style="display:flex;gap:8px;margin-bottom:12px;">
    <button id="btn-bars" class="ts-controls" onclick="showView('bars')"
            style="padding:4px 12px;border:1px solid
#E5E7EB;border-radius:6px;font-size:11px;cursor:pointer;background:#0066FF;color:white;">Ba
rs</button>
    <button id="btn-radar" class="ts-controls" onclick="showView('radar')"
            style="padding:4px 12px;border:1px solid
#E5E7EB;border-radius:6px;font-size:11px;cursor:pointer;background:white;">Radar</button>
  </div>

  <!-- Domain Bars -->
  <div id="domain-bars" class="domain-section">
    <h3>Domain Breakdown</h3>
    <div id="domain-bar-list"></div>
  </div>

  <!-- Radar Chart -->
  <div id="domain-radar" class="radar-container" style="display:none;"></div>

  <!-- Key Insights -->
  <div id="insights" class="insights"></div>

  <!-- Footer -->
  <div class="vc-footer">
    Data: <a href="https://visualclimate.org" target="_blank">visualclimate.org</a> |
    Sources: World Bank, OWID, Ember, ND-GAIN, Climate TRACE
  </div>
</div>

<!-- Tableau Extensions API -->
<script
src="https://cdn.jsdelivr.net/npm/@anthropic-ai/tableau-extensions-api@latest/lib/tableau.extens
ions.1.latest.js"></script>
<!-- 주의: 위 CDN이 안 되면 아래 공식 경로 사용 -->
```

```
  <!-- <script
src="https://tableau.github.io/extensions-api/lib/tableau.extensions.1.latest.js"></script> -->

  <script src="shared.js"></script>
  <script src="radar-chart.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

---

6. `extension/app.js` — Extension 1: Climate Card 로직

```
/**
 * app.js — Visual Climate Climate Card Extension
 *
 * 동작 흐름:
 * 1. tableau.extensions.initializeAsync()
 * 2. settings에서 Supabase URL/Key 로드
 *    → 없으면 setup screen 표시 → 사용자 입력 → settings에 저장
 * 3. 대시보드 워크시트에서 ISO3 탐지 (필터 또는 마크 선택)
 * 4. Supabase REST API로 country + REPORT.* 지표 조회
 * 5. Climate Card UI 렌더링
 * 6. FilterChanged / MarkSelectionChanged 이벤트로 자동 업데이트
 */

(function () {
  'use strict';

  let currentISO3 = null;

  // ─── 초기화 ───
  document.addEventListener('DOMContentLoaded', function () {
    if (typeof tableau === 'undefined' || !tableau.extensions) {
      // Tableau 외부 테스트 모드
      console.warn('Tableau Extensions API not available. Running in standalone mode.');
      showSetupScreen();
      return;
    }

    tableau.extensions.initializeAsync({ configure: showConfigDialog }).then(function () {
      VC.loadConfigFromSettings();

      const settings = tableau.extensions.settings;
```

```
    const url = settings.get('supabaseUrl');
    const key = settings.get('supabaseKey');

    if (!url || !key) {
      showSetupScreen();
    } else {
      VC.configureSupabase(url, key);
      startExtension();
    }
  }).catch(function (err) {
    console.error('Init error:', err);
    showError('Extension initialization failed: ' + err.message);
  });
});

// ─── Setup Screen ───
function showSetupScreen() {
  hideAll();
  document.getElementById('setup-screen').style.display = 'flex';
  document.getElementById('btn-save-config').addEventListener('click', onSaveConfig);
}

async function onSaveConfig() {
  const url = document.getElementById('input-url').value.trim();
  const key = document.getElementById('input-key').value.trim();
  const errEl = document.getElementById('setup-error');

  if (!url || !key) {
    errEl.textContent = 'Both URL and Key are required.';
    return;
  }

  // 연결 테스트
  VC.configureSupabase(url, key);
  try {
    const test = await VC.supabaseGet('countries', 'limit=1&select=iso3');
    if (!test || test.length === 0) throw new Error('No data returned');
  } catch (e) {
    errEl.textContent = 'Connection failed: ' + e.message;
    return;
  }

  // 저장
  if (typeof tableau !== 'undefined' && tableau.extensions) {
```

```
    await VC.saveConfigToSettings(url, key);
  }

  startExtension();
}

function showConfigDialog() {
  showSetupScreen();
}

// ─── Extension 시작 ───
async function startExtension() {
  hideAll();
  document.getElementById('loading').style.display = 'flex';

  try {
    const iso3 = await VC.detectISO3FromDashboard();
    if (iso3) {
      await renderCard(iso3);
    } else {
      showEmptyState();
    }
  } catch (e) {
    console.error('Start error:', e);
    showEmptyState();
  }

  // 이벤트 리스너
  VC.onFilterChange(onDashboardChange);
  VC.onMarkSelection(onDashboardChange);
}

async function onDashboardChange() {
  try {
    const iso3 = await VC.detectISO3FromDashboard();
    if (iso3 && iso3 !== currentISO3) {
      await renderCard(iso3);
    } else if (!iso3) {
      showEmptyState();
    }
  } catch (e) {
    console.error('Dashboard change error:', e);
  }
}
```

```
// ─── Climate Card 렌더링 ───
async function renderCard(iso3) {
  hideAll();
  document.getElementById('loading').style.display = 'flex';
  currentISO3 = iso3;

  try {
    // 병렬 조회
    const [card, peer] = await Promise.all([
      VC.getReportCard(iso3),
      VC.getPeerContext(iso3),
    ]);

    if (!card) {
      showEmptyState();
      return;
    }

    hideAll();
    document.getElementById('climate-card').style.display = 'block';

    // Header
    document.getElementById('card-flag').textContent = VC.iso3ToFlag(iso3);
    document.getElementById('card-name').textContent = card.name;
    document.getElementById('card-meta').textContent =
      `${iso3} · ${card.region || '—'} · ${card.income_group || '—'}`;

    // Score Hero
    document.getElementById('card-score').textContent =
      card.total_score != null ? VC.fmt(card.total_score) : '—';

    const gradeEl = document.getElementById('card-grade');
    const grade = card.grade || '—';
    gradeEl.textContent = grade;
    gradeEl.style.background = VC.GRADE_BG[grade] || '#F8F9FA';
    gradeEl.style.color = VC.GRADE_COLOR[grade] || '#1A1A2E';

    // Class Pill
    const classEl = document.getElementById('card-class');
    const className = VC.CLASS_LABEL[card.climate_class] || null;
    if (className) {
      classEl.textContent = className;
      classEl.style.background = VC.CLASS_COLOR[className];
```

```
      classEl.style.display = 'inline-block';
    } else {
      classEl.style.display = 'none';
    }

    // Rank
    document.getElementById('card-rank').textContent =
      peer.globalRank ? `#${peer.globalRank} / ${peer.totalCountries}` : '';

    // Domain Bars
    renderDomainBars(card);

    // Radar Chart
    RadarChart.render('domain-radar', {
      emissions: card.emissions_score,
      energy: card.energy_score,
      economy: card.economy_score,
      responsibility: card.responsibility_score,
      resilience: card.resilience_score,
    });

    // Key Insights
    renderInsights(card, peer);

  } catch (e) {
    console.error('Render error:', e);
    showError('Failed to load data for ' + iso3);
  }
}

function renderDomainBars(card) {
  const container = document.getElementById('domain-bar-list');
  container.innerHTML = '';

  // 글로벌 평균 (대략적 중앙값; 실제 계산은 getPeerContext에서 하되 여기서는 시각 참조용)
  // RAG 레포에서의 global avg는 약: Emissions 55, Energy 50, Economy 45, Responsibility
60, Resilience 40
  const GLOBAL_AVG = { emissions: 55, energy: 50, economy: 45, responsibility: 60,
resilience: 40 };

  for (const d of VC.DOMAINS) {
    const score = card[d.scoreField];
    const pct = score != null ? Math.min(100, Math.max(0, score)) : 0;
    const avg = GLOBAL_AVG[d.key] || 50;
```

```
      const row = document.createElement('div');
      row.className = 'domain-bar-row';
      row.innerHTML = `
        <span class="domain-label">${d.label} <span
style="font-size:9px;color:${VC.BRAND.textMuted}">(${d.weight})</span></span>
        <div class="domain-bar-track">
          <div class="domain-bar-fill" style="width:${pct}%;background:${d.color};"></div>
          <div class="domain-bar-avg" style="left:${avg}%;" title="Global Avg: ${avg}"></div>
        </div>
        <span class="domain-bar-value" style="color:${score != null ? d.color :
VC.BRAND.dataMissing}">${score != null ? VC.fmt(score) : '—'}</span>
      `;
      container.appendChild(row);
    }
  }

  function renderInsights(card, peer) {
    const el = document.getElementById('insights');
    const lines = [];

    // 최고/최저 도메인 (ReportCardClient.tsx 로직)
    const domainScores = VC.DOMAINS
      .map(d => ({ label: d.label, score: card[d.scoreField] }))
      .filter(d => d.score != null)
      .sort((a, b) => b.score - a.score);

    if (domainScores.length > 0) {
      const best = domainScores[0];
      const worst = domainScores[domainScores.length - 1];
      lines.push(`<strong>Strongest:</strong> ${best.label} (${VC.fmt(best.score)},
${VC.perfLabel(best.score)})`);
      if (domainScores.length > 1) {
        lines.push(`<strong>Weakest:</strong> ${worst.label} (${VC.fmt(worst.score)},
${VC.perfLabel(worst.score)})`);
      }
    }

    // Climate Class 설명
    const className = VC.CLASS_LABEL[card.climate_class];
    if (className && VC.CLASS_EXPLAIN[className]) {
      lines.push(`<strong>Classification:</strong> ${className} —
${VC.CLASS_EXPLAIN[className]}`);
    }
```

```
    // 순위 컨텍스트
    if (peer.globalRank) {
      lines.push(`<strong>Global Rank:</strong> #${peer.globalRank} of ${peer.totalCountries}
countries`);
    }
    if (peer.incomeRank) {
      lines.push(`<strong>Income Group Rank:</strong> #${peer.incomeRank} of
${peer.incomeTotal}`);
    }

    el.innerHTML = lines.join('<br>');
  }

  // ─── View Toggle ───
  window.showView = function (view) {
    const barsEl = document.getElementById('domain-bars');
    const radarEl = document.getElementById('domain-radar');
    const btnBars = document.getElementById('btn-bars');
    const btnRadar = document.getElementById('btn-radar');

    if (view === 'radar') {
      barsEl.style.display = 'none';
      radarEl.style.display = 'flex';
      btnBars.style.background = 'white'; btnBars.style.color = '#1A1A2E';
      btnRadar.style.background = '#0066FF'; btnRadar.style.color = 'white';
    } else {
      barsEl.style.display = 'block';
      radarEl.style.display = 'none';
      btnBars.style.background = '#0066FF'; btnBars.style.color = 'white';
      btnRadar.style.background = 'white'; btnRadar.style.color = '#1A1A2E';
    }
  };

  // ─── UI 헬퍼 ───
  function hideAll() {
    document.getElementById('setup-screen').style.display = 'none';
    document.getElementById('loading').style.display = 'none';
    document.getElementById('empty-state').style.display = 'none';
    document.getElementById('climate-card').style.display = 'none';
  }

  function showEmptyState() {
    hideAll();
```

```
    document.getElementById('empty-state').style.display = 'flex';
  }

  function showError(msg) {
    hideAll();
    const el = document.getElementById('empty-state');
    el.style.display = 'flex';
    el.innerHTML = `<p style="color:#E5484D">${msg}</p>`;
  }
})();
```

---

7. `extension/timeseries.trex` — Timeseries Explorer Manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest manifest-version="0.1" xmlns="http://www.tableau.com/xml/extension_manifest">
  <dashboard-extension id="com.visualclimate.timeseries-explorer" extension-version="1.0.0">
    <default-locale>en_US</default-locale>
    <name>Visual Climate — Timeseries Explorer</name>
    <description>
      Interactive time-series charts for 61 climate indicators across 250 countries.
      Connects to Supabase country_data table (172,121 rows).
      Supports emissions, energy, sector, fuel breakdown, resilience, and economy indicators.
    </description>
    <author name="Harim Jung" email="harim@visualclimate.org"
            organization="Visual Climate" website="https://visualclimate.org"/>
    <min-api-version>1.4</min-api-version>
    <source-location>
      <url>http://localhost:8765/extension/timeseries.html</url>
    </source-location>

<icon>iVBORw0KGgoAAAANSUhEUgAAABgAAAAYCAYAAADgdz34AAAABmJLR0QA/wD/AP
+gvaeTAAAASkIEQVRIie2UMQ4AIAgE+f+nsdBYYIOxcRqchhMQJEnS30TETkPkMbABJhYi4h
m9BpjktcQWN7JDJDlHkq0LFiR+I0mSvuMCyJYZOXCe/VQAAAAASUVORK5CYII=</icon>
    <permissions>
      <permission>full data</permission>
    </permissions>
    <context-menu>
      <configure-context-menu-item />
    </context-menu>
  </dashboard-extension>
```

```
</manifest>
```

---

8. `extension/timeseries.html` — Extension 2: Timeseries Explorer

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Visual Climate — Timeseries Explorer</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <div id="setup-screen" class="setup-screen" style="display:none;">
    <h2>Visual Climate — Timeseries Explorer</h2>
    <p style="margin-bottom:16px;color:#4A4A6A;font-size:12px;">Supabase 연결 정보 입력</p>
    <input type="text" id="input-url" placeholder="Supabase URL">
    <input type="text" id="input-key" placeholder="Supabase Key">
    <button id="btn-save-config">Connect</button>
    <p id="setup-error" style="color:#E5484D;font-size:11px;margin-top:8px;"></p>
  </div>

  <div id="loading" class="loading" style="display:none;">
    <div class="spinner"></div>
    <span>Loading time-series data...</span>
  </div>

  <div id="empty-state" class="empty-state" style="display:none;">
    <p>Select a country to explore its climate time-series.</p>
  </div>

  <div id="ts-main" style="display:none;">
    <!-- Country Header (compact) -->
    <div style="padding:12px 16px;display:flex;align-items:center;gap:8px;border-bottom:1px
solid #E5E7EB;">
      <span id="ts-flag" style="font-size:24px;"></span>
      <span id="ts-name" style="font-weight:600;font-size:15px;"></span>
      <span id="ts-iso3" style="font-size:11px;color:#8888A0;"></span>
    </div>
```

```
    <!-- Controls -->
    <div class="ts-controls" style="padding:12px 16px;">
      <select id="ts-group">
        <option value="emissions">Emissions</option>
        <option value="fuel">Fuel Breakdown</option>
        <option value="sector">Sector (Climate TRACE)</option>
        <option value="energy">Energy</option>
        <option value="economy">Economy</option>
        <option value="resilience">Resilience</option>
        <option value="other_ghg">Other GHG</option>
      </select>
      <select id="ts-indicator"></select>
    </div>

    <!-- Chart Area (pure SVG) -->
    <div id="ts-chart" class="ts-chart-area"></div>

    <!-- Data Table -->
    <div id="ts-table" style="padding:8px 16px;overflow-x:auto;"></div>

    <div class="vc-footer">
      Data: <a href="https://visualclimate.org" target="_blank">visualclimate.org</a> Supabase
    </div>
  </div>

  <script src="https://tableau.github.io/extensions-api/lib/tableau.extensions.1.latest.js"></script>
  <script src="shared.js"></script>
  <script src="timeseries-app.js"></script>
</body>
</html>
```

---

9. `extension/timeseries-app.js` — Extension 2: Timeseries 로직

```
/**
 * timeseries-app.js — Visual Climate Timeseries Explorer Extension
 *
 * Supabase country_data 테이블에서 시계열을 직접 조회.
 * CountryClient.tsx의 실제 indicator_code 기반.
 *
```

```
 * 지원 그룹:
 *   emissions: EN.GHG.CO2.PC.CE.AR5, OWID.GHG_PER_CAPITA, OWID.CO2, etc.
 *   fuel: OWID.COAL_CO2, OWID.OIL_CO2, OWID.GAS_CO2, etc.
 *   sector: CTRACE.POWER, CTRACE.TRANSPORTATION, etc.
 *   energy: EMBER.RENEWABLE.PCT, EMBER.CARBON.INTENSITY, EG.FEC.RNEW.ZS
 *   economy: NY.GDP.PCAP.CD, SP.POP.TOTL, DERIVED.CO2_PER_GDP
 *   resilience: NDGAIN.VULNERABILITY, NDGAIN.READINESS
 *   other_ghg: OWID.METHANE, OWID.NITROUS_OXIDE
 */

(function () {
  'use strict';

  let currentISO3 = null;
  let currentData = []; // [{indicator_code, year, value, source}]

  // ─── Indicator 라벨 매핑 (schema.json + pull-from-supabase.ts 기반) ───
  const CODE_LABELS = {
    'EN.GHG.CO2.PC.CE.AR5': 'CO₂ per capita (WB, tCO₂/cap)',
    'OWID.GHG_PER_CAPITA': 'GHG per capita (OWID, tCO₂eq/cap)',
    'OWID.CO2': 'CO₂ fossil (OWID, MtCO₂)',
    'OWID.TOTAL_GHG_EXCLUDING_LUCF': 'Total GHG excl. LULUCF (MtCO₂eq)',
    'OWID.CONSUMPTION_CO2_PER_CAPITA': 'Consumption CO₂/cap (tCO₂)',
    'OWID.CO2_PER_GDP': 'CO₂ per GDP (kgCO₂/$)',
    'OWID.COAL_CO2': 'Coal CO₂ (MtCO₂)',
    'OWID.OIL_CO2': 'Oil CO₂ (MtCO₂)',
    'OWID.GAS_CO2': 'Gas CO₂ (MtCO₂)',
    'OWID.CEMENT_CO2': 'Cement CO₂ (MtCO₂)',
    'OWID.FLARING_CO2': 'Flaring CO₂ (MtCO₂)',
    'CTRACE.POWER': 'Power sector (MtCO₂eq)',
    'CTRACE.TRANSPORTATION': 'Transportation (MtCO₂eq)',
    'CTRACE.MANUFACTURING': 'Manufacturing (MtCO₂eq)',
    'CTRACE.AGRICULTURE': 'Agriculture (MtCO₂eq)',
    'CTRACE.BUILDINGS': 'Buildings (MtCO₂eq)',
    'CTRACE.WASTE': 'Waste (MtCO₂eq)',
    'CTRACE.MINERAL_EXTRACTION': 'Mineral extraction (MtCO₂eq)',
    'CTRACE.FLUORINATED_GASES': 'F-gases (MtCO₂eq)',
    'EMBER.RENEWABLE.PCT': 'Renewable electricity share (%)',
    'EMBER.CARBON.INTENSITY': 'Grid carbon intensity (gCO₂/kWh)',
    'EG.FEC.RNEW.ZS': 'Renewable final energy share (%, WB)',
    'NY.GDP.PCAP.CD': 'GDP per capita (USD)',
    'SP.POP.TOTL': 'Population',
    'DERIVED.CO2_PER_GDP': 'CO₂ per GDP (derived)',
    'NDGAIN.VULNERABILITY': 'ND-GAIN Vulnerability (0-1)',
```

```
  'NDGAIN.READINESS': 'ND-GAIN Readiness (0-1)',
  'OWID.METHANE': 'Methane (MtCO₂eq)',
  'OWID.NITROUS_OXIDE': 'N₂O (MtCO₂eq)',
};

// ─── 초기화 ───
document.addEventListener('DOMContentLoaded', function () {
  if (typeof tableau === 'undefined' || !tableau.extensions) {
    showSetupScreen();
    return;
  }

  tableau.extensions.initializeAsync({ configure: showSetupScreen }).then(function () {
    VC.loadConfigFromSettings();
    const settings = tableau.extensions.settings;
    const url = settings.get('supabaseUrl');
    const key = settings.get('supabaseKey');

    if (!url || !key) {
      showSetupScreen();
    } else {
      VC.configureSupabase(url, key);
      startExtension();
    }
  });
});

function showSetupScreen() {
  hideAll();
  document.getElementById('setup-screen').style.display = 'flex';
  document.getElementById('btn-save-config').addEventListener('click', async function () {
    const url = document.getElementById('input-url').value.trim();
    const key = document.getElementById('input-key').value.trim();
    if (!url || !key) {
      document.getElementById('setup-error').textContent = 'Both fields required.';
      return;
    }
    VC.configureSupabase(url, key);
    try {
      await VC.supabaseGet('countries', 'limit=1&select=iso3');
    } catch (e) {
      document.getElementById('setup-error').textContent = 'Connection failed: ' + e.message;
      return;
    }
```

```
    if (typeof tableau !== 'undefined' && tableau.extensions) {
      await VC.saveConfigToSettings(url, key);
    }
    startExtension();
  });
}

async function startExtension() {
  hideAll();
  document.getElementById('loading').style.display = 'flex';

  // 그룹/지표 드롭다운 설정
  const groupSelect = document.getElementById('ts-group');
  const indicatorSelect = document.getElementById('ts-indicator');

  groupSelect.addEventListener('change', onGroupChange);
  indicatorSelect.addEventListener('change', onIndicatorChange);

  try {
    const iso3 = await VC.detectISO3FromDashboard();
    if (iso3) {
      await loadCountry(iso3);
    } else {
      showEmptyState();
    }
  } catch (e) {
    showEmptyState();
  }

  VC.onFilterChange(onDashboardChange);
  VC.onMarkSelection(onDashboardChange);
}

async function onDashboardChange() {
  const iso3 = await VC.detectISO3FromDashboard();
  if (iso3 && iso3 !== currentISO3) {
    await loadCountry(iso3);
  }
}

async function loadCountry(iso3) {
  hideAll();
  document.getElementById('loading').style.display = 'flex';
  currentISO3 = iso3;
```

```
  const country = await VC.getCountry(iso3);
  if (!country) { showEmptyState(); return; }

  // 현재 그룹의 코드로 시계열 조회
  const group = document.getElementById('ts-group').value;
  const codes = VC.TIMESERIES_GROUPS[group]?.codes || [];

  currentData = await VC.getMultiTimeseries(iso3, codes);

  hideAll();
  document.getElementById('ts-main').style.display = 'block';
  document.getElementById('ts-flag').textContent = VC.iso3ToFlag(iso3);
  document.getElementById('ts-name').textContent = country.name;
  document.getElementById('ts-iso3').textContent = iso3;

  populateIndicatorSelect(codes);
  renderChart();
}

function onGroupChange() {
  const group = document.getElementById('ts-group').value;
  const codes = VC.TIMESERIES_GROUPS[group]?.codes || [];
  populateIndicatorSelect(codes);

  // 새 그룹의 데이터 로드
  if (currentISO3) {
    document.getElementById('loading').style.display = 'flex';
    VC.getMultiTimeseries(currentISO3, codes).then(function (data) {
      currentData = data;
      document.getElementById('loading').style.display = 'none';
      renderChart();
    });
  }
}

function onIndicatorChange() {
  renderChart();
}

function populateIndicatorSelect(codes) {
  const sel = document.getElementById('ts-indicator');
  sel.innerHTML = '';
```

```
    // "All" 옵션
    const allOpt = document.createElement('option');
    allOpt.value = '__ALL__';
    allOpt.textContent = '— All indicators —';
    sel.appendChild(allOpt);

    // 데이터가 있는 코드만 표시
    const availableCodes = new Set(currentData.map(r => r.indicator_code));
    for (const code of codes) {
      if (availableCodes.has(code)) {
        const opt = document.createElement('option');
        opt.value = code;
        opt.textContent = CODE_LABELS[code] || code;
        sel.appendChild(opt);
      }
    }
  }

  // ─── SVG 라인 차트 렌더링 ───
  function renderChart() {
    const container = document.getElementById('ts-chart');
    const selectedCode = document.getElementById('ts-indicator').value;

    // 데이터 필터링
    let filtered = currentData;
    if (selectedCode !== '__ALL__') {
      filtered = currentData.filter(r => r.indicator_code === selectedCode);
    }

    if (filtered.length === 0) {
      container.innerHTML = '<div class="empty-state"><p>No data available for this
indicator.</p></div>';
      renderTable([]);
      return;
    }

    // 코드별 그룹핑
    const groups = {};
    for (const row of filtered) {
      if (!groups[row.indicator_code]) groups[row.indicator_code] = [];
      groups[row.indicator_code].push(row);
    }

    // 차트 크기
```

```
  const W = container.clientWidth || 600;
  const H = container.clientHeight || 350;
  const PAD = { top: 20, right: 20, bottom: 40, left: 60 };
  const plotW = W - PAD.left - PAD.right;
  const plotH = H - PAD.top - PAD.bottom;

  // 전체 연도/값 범위
  const allYears = filtered.map(r => r.year);
  const allValues = filtered.map(r => r.value);
  const minYear = Math.min(...allYears);
  const maxYear = Math.max(...allYears);
  const minVal = Math.min(0, Math.min(...allValues));
  const maxVal = Math.max(...allValues) * 1.1;

  const xScale = (year) => PAD.left + ((year - minYear) / (maxYear - minYear || 1)) * plotW;
  const yScale = (val) => PAD.top + plotH - ((val - minVal) / (maxVal - minVal || 1)) * plotH;

  let svg = `<svg width="${W}" height="${H}" viewBox="0 0 ${W} ${H}"
xmlns="http://www.w3.org/2000/svg">`;

  // 격자선
  const yTicks = 5;
  for (let i = 0; i <= yTicks; i++) {
    const v = minVal + (maxVal - minVal) * (i / yTicks);
    const y = yScale(v);
    svg += `<line x1="${PAD.left}" y1="${y}" x2="${W - PAD.right}" y2="${y}" stroke="#E5E7EB"
stroke-width="0.5"/>`;
    svg += `<text x="${PAD.left - 6}" y="${y + 3}" text-anchor="end" font-size="9" fill="#8888A0"
font-family="JetBrains Mono,monospace">${formatTickVal(v)}</text>`;
  }

  // X축 연도
  const yearStep = Math.max(1, Math.floor((maxYear - minYear) / 8));
  for (let y = minYear; y <= maxYear; y += yearStep) {
    const x = xScale(y);
    svg += `<text x="${x}" y="${H - 8}" text-anchor="middle" font-size="9" fill="#8888A0"
font-family="JetBrains Mono,monospace">${y}</text>`;
  }

  // 각 지표의 라인
  const COLORS = ['#0066FF', '#E5484D', '#00A67E', '#F59E0B', '#8B5CF6', '#FF6B6B',
'#4ECDC4', '#45B7D1'];
  let colorIdx = 0;
```

```
    for (const [code, rows] of Object.entries(groups)) {
      const sorted = rows.sort((a, b) => a.year - b.year);
      const color = COLORS[colorIdx % COLORS.length];
      colorIdx++;

      // 라인 경로
      const pathParts = sorted.map((r, i) => {
        const x = xScale(r.year);
        const y = yScale(r.value);
        return i === 0 ? `M ${x} ${y}` : `L ${x} ${y}`;
      });
      svg += `<path d="${pathParts.join(' ')}" fill="none" stroke="${color}" stroke-width="2"/>`;

      // 마지막 점 + 라벨
      const last = sorted[sorted.length - 1];
      const lx = xScale(last.year);
      const ly = yScale(last.value);
      svg += `<circle cx="${lx}" cy="${ly}" r="3" fill="${color}"/>`;

      const shortLabel = (CODE_LABELS[code] || code).substring(0, 20);
      svg += `<text x="${lx + 6}" y="${ly + 3}" font-size="8" fill="${color}"
font-family="Inter,sans-serif">${shortLabel}</text>`;
    }

    svg += '</svg>';
    container.innerHTML = svg;

    // 테이블
    renderTable(filtered);
  }

  function renderTable(data) {
    const container = document.getElementById('ts-table');
    if (data.length === 0) {
      container.innerHTML = '';
      return;
    }

    // 최근 10행만 (최신순)
    const sorted = [...data].sort((a, b) => b.year - a.year ||
a.indicator_code.localeCompare(b.indicator_code));
    const recent = sorted.slice(0, 20);

    let html = '<table style="width:100%;font-size:10px;border-collapse:collapse;">';
```

```
    html += '<tr style="border-bottom:1px solid #E5E7EB;"><th
style="text-align:left;padding:3px;">Indicator</th><th>Year</th><th
style="text-align:right;">Value</th><th>Source</th></tr>';
    for (const r of recent) {
      const label = (CODE_LABELS[r.indicator_code] || r.indicator_code).substring(0, 35);
      html += `<tr><td style="padding:3px;">${label}</td><td
style="text-align:center;">${r.year}</td><td style="text-align:right;font-family:JetBrains
Mono,monospace;">${VC.fmt(r.value, 2)}</td><td style="color:#8888A0;">${r.source ||
''}</td></tr>`;
    }
    html += '</table>';
    container.innerHTML = html;
  }

  function formatTickVal(v) {
    if (Math.abs(v) >= 1e6) return (v / 1e6).toFixed(1) + 'M';
    if (Math.abs(v) >= 1e3) return (v / 1e3).toFixed(1) + 'K';
    if (Math.abs(v) < 0.01) return v.toFixed(3);
    return v.toFixed(1);
  }

  // ─── UI 헬퍼 ───
  function hideAll() {
    ['setup-screen', 'loading', 'empty-state', 'ts-main'].forEach(function (id) {
      document.getElementById(id).style.display = 'none';
    });
  }
  function showEmptyState() {
    hideAll();
    document.getElementById('empty-state').style.display = 'flex';
  }
})();
```

---

10. `extension/ndc-tracker.trex` — NDC Gap Tracker Manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest manifest-version="0.1" xmlns="http://www.tableau.com/xml/extension_manifest">
  <dashboard-extension id="com.visualclimate.ndc-gap-tracker" extension-version="1.0.0">
    <default-locale>en_US</default-locale>
    <name>Visual Climate — NDC Gap Tracker</name>
```

```
    <description>
      Tracks the gap between actual emissions trajectory and NDC 2030/2035 targets.
      Covers 20 major emitters with targets from UNFCCC NDC Registry, Climate Watch, and
CAT.
      Data: ndc-targets-v2.json + Supabase country_data time-series.
    </description>
    <author name="Harim Jung" email="harim@visualclimate.org"
          organization="Visual Climate" website="https://visualclimate.org"/>
    <min-api-version>1.4</min-api-version>
    <source-location>
      <url>http://localhost:8765/extension/ndc-tracker.html</url>
    </source-location>

<icon>iVBORw0KGgoAAAANSUhEUgAAABgAAAAYCAYAAADgdz34AAAABmJLR0QA/wD/AP
+gvaeTAAAASkIEQVRIie2UMQ4AIAgE+f+nsdBYYIOxcRqchhMQJEnS30TETkPkMbABJhYi4h
m9BpjktcQWN7JDJDlHkq0LFiR+I0mSvuMCyJYZOXCe/VQAAAAASUVORK5CYII=</icon>
    <permissions>
      <permission>full data</permission>
    </permissions>
    <context-menu>
      <configure-context-menu-item />
    </context-menu>
  </dashboard-extension>
</manifest>
```

---

11. `extension/ndc-tracker.html` — Extension 3: NDC Gap Tracker

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Visual Climate — NDC Gap Tracker</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <div id="setup-screen" class="setup-screen" style="display:none;">
    <h2>Visual Climate — NDC Gap Tracker</h2>
    <p style="margin-bottom:16px;color:#4A4A6A;font-size:12px;">Supabase 연결 정보 입력</p>
```

```
    <input type="text" id="input-url" placeholder="Supabase URL">
    <input type="text" id="input-key" placeholder="Supabase Key">
    <button id="btn-save-config">Connect</button>
    <p id="setup-error" style="color:#E5484D;font-size:11px;margin-top:8px;"></p>
  </div>

  <div id="loading" class="loading" style="display:none;">
    <div class="spinner"></div>
    <span>Loading NDC target data...</span>
  </div>

  <div id="empty-state" class="empty-state" style="display:none;">
    <p>Select one of the 20 tracked countries to view its NDC gap analysis.</p>
    <p style="font-size:11px;color:#8888A0;margin-top:8px;">
      Covered: KOR, USA, DEU, CHN, IND, JPN, GBR, BRA, CAN, AUS, FRA, IDN, NGA, BGD,
ZAF, MEX, RUS, SAU, TUR, EGY
    </p>
  </div>

  <div id="ndc-main" class="ndc-card" style="display:none;">
    <!-- Header -->
    <div class="ndc-header">
      <div style="display:flex;align-items:center;gap:8px;margin-bottom:8px;">
        <span id="ndc-flag" style="font-size:28px;"></span>
        <div>
          <div id="ndc-name" style="font-weight:700;font-size:16px;"></div>
          <div id="ndc-cat" style="font-size:11px;"></div>
        </div>
      </div>
    </div>

    <!-- KPI Row -->
    <div class="ndc-kpi-row">
      <div class="ndc-kpi">
        <div id="kpi-target" class="ndc-kpi-value"></div>
        <div class="ndc-kpi-label">NDC 2030 Target</div>
      </div>
      <div class="ndc-kpi">
        <div id="kpi-nz" class="ndc-kpi-value"></div>
        <div class="ndc-kpi-label">Net-Zero Year</div>
      </div>
      <div class="ndc-kpi">
        <div id="kpi-countdown" class="ndc-kpi-value"></div>
        <div class="ndc-kpi-label">NZ Countdown</div>
```

```
    </div>
  </div>

  <!-- Gap Badge -->
  <div style="text-align:center;margin-bottom:12px;">
    <span id="ndc-gap-badge" class="ndc-gap-badge"></span>
  </div>

  <!-- Emissions Trajectory Chart -->
  <div id="ndc-chart" class="ndc-chart-area" style="height:280px;"></div>

  <!-- NDC 3.0 Info -->
  <div id="ndc3-info" style="display:none;padding:8px
12px;background:#F8F9FA;border-radius:6px;margin-bottom:12px;font-size:11px;">
  </div>

  <!-- Detail Table -->
  <table id="ndc-detail" class="ndc-detail" style="width:100%;font-size:11px;">
  </table>

  <div class="vc-footer">
    Sources: UNFCCC NDC Registry, Climate Watch, CAT |
    <a href="https://visualclimate.org" target="_blank">visualclimate.org</a>
  </div>
 </div>

 <script src="https://tableau.github.io/extensions-api/lib/tableau.extensions.1.latest.js"></script>
 <script src="shared.js"></script>
 <script src="ndc-tracker-app.js"></script>
</body>
</html>
```

---

12. `extension/ndc-tracker-app.js` — Extension 3: NDC Gap Tracker 로직

```
/**
 * ndc-tracker-app.js — Visual Climate NDC Gap Tracker Extension
 *
 * ndc-targets-v2.json의 20국 데이터 + Supabase 실배출 시계열로
 * 실제 배출 궤적 vs NDC 목표를 시각화.
 *
```

```
 * 갭 계산식 (calculated-fields.md CF04/CF05 기반):
 *   CF04 = projected_2030 - ndc_2030_target
 *   CF05 = On Track / Narrow Gap / Significant Gap / Critical Gap
 *
 * 실배출 데이터 소스:
 *   OWID.TOTAL_GHG_EXCLUDING_LUCF (MtCO₂eq) — 가장 포괄적
 *   fallback: OWID.CO2 (MtCO₂) — CO₂만
 */

(function () {
  'use strict';

  let currentISO3 = null;

  // ─── 초기화 ───
  document.addEventListener('DOMContentLoaded', function () {
    if (typeof tableau === 'undefined' || !tableau.extensions) {
      showSetupScreen();
      return;
    }

    tableau.extensions.initializeAsync({ configure: showSetupScreen }).then(function () {
      VC.loadConfigFromSettings();
      const settings = tableau.extensions.settings;
      const url = settings.get('supabaseUrl');
      const key = settings.get('supabaseKey');

      if (!url || !key) {
        showSetupScreen();
      } else {
        VC.configureSupabase(url, key);
        startExtension();
      }
    });
  });

  function showSetupScreen() {
    hideAll();
    document.getElementById('setup-screen').style.display = 'flex';
    document.getElementById('btn-save-config').addEventListener('click', async function () {
      const url = document.getElementById('input-url').value.trim();
      const key = document.getElementById('input-key').value.trim();
      if (!url || !key) {
        document.getElementById('setup-error').textContent = 'Both fields required.';
```

```
      return;
    }
    VC.configureSupabase(url, key);
    try { await VC.supabaseGet('countries', 'limit=1&select=iso3'); }
    catch (e) { document.getElementById('setup-error').textContent = 'Failed: ' + e.message;
return; }
    if (typeof tableau !== 'undefined' && tableau.extensions) {
      await VC.saveConfigToSettings(url, key);
    }
    startExtension();
  });
}

async function startExtension() {
  hideAll();
  document.getElementById('loading').style.display = 'flex';

  try {
    const iso3 = await VC.detectISO3FromDashboard();
    if (iso3 && VC.NDC_TARGETS[iso3]) {
      await renderNDC(iso3);
    } else if (iso3) {
      showNotCovered(iso3);
    } else {
      showEmptyState();
    }
  } catch (e) {
    showEmptyState();
  }

  VC.onFilterChange(onDashboardChange);
  VC.onMarkSelection(onDashboardChange);
}

async function onDashboardChange() {
  const iso3 = await VC.detectISO3FromDashboard();
  if (iso3 && iso3 !== currentISO3) {
    if (VC.NDC_TARGETS[iso3]) {
      await renderNDC(iso3);
    } else {
      showNotCovered(iso3);
    }
  }
}
```

```
// ─── NDC Tracker 렌더링 ───
async function renderNDC(iso3) {
  hideAll();
  document.getElementById('loading').style.display = 'flex';
  currentISO3 = iso3;

  const target = VC.NDC_TARGETS[iso3];
  const country = await VC.getCountry(iso3);

  // 실배출 시계열 조회
  let emissionsTS = await VC.getTimeseries(iso3, 'OWID.TOTAL_GHG_EXCLUDING_LUCF');
  if (emissionsTS.length === 0) {
    emissionsTS = await VC.getTimeseries(iso3, 'OWID.CO2');
  }

  hideAll();
  document.getElementById('ndc-main').style.display = 'block';

  // Header
  document.getElementById('ndc-flag').textContent = VC.iso3ToFlag(iso3);
  document.getElementById('ndc-name').textContent = country?.name || target.name;

  // CAT Rating
  const catEl = document.getElementById('ndc-cat');
  if (target.cat) {
    const catColor = VC.CAT_COLOR[target.cat] || '#CCCCCC';
    const catLabel = target.cat.replace(/_/g, ' ');
    catEl.innerHTML = `CAT: <span
style="color:${catColor};font-weight:600;">${catLabel}</span>`;
  } else {
    catEl.textContent = 'CAT: Not rated';
  }

  // KPIs
  document.getElementById('kpi-target').textContent = `-${target.ndc2030_pct}%`;
  document.getElementById('kpi-target').style.color = VC.BRAND.primary;

  document.getElementById('kpi-nz').textContent = target.nz_year || '—';
  document.getElementById('kpi-nz').style.color = target.nz_year ? VC.BRAND.textPrimary :
VC.BRAND.dataMissing;

  // Net Zero Countdown (CF08: nz_year - current year)
  const currentYear = new Date().getFullYear();
```

```
  const countdown = target.nz_year ? (target.nz_year - currentYear) : null;
  document.getElementById('kpi-countdown').textContent = countdown != null ?
`${countdown}yr` : '—';

  // Gap 계산
  const gapInfo = calculateGap(emissionsTS, target);
  const gapBadge = document.getElementById('ndc-gap-badge');
  gapBadge.textContent = gapInfo.severity;
  gapBadge.style.background = VC.GAP_COLOR[gapInfo.severity] || '#CCCCCC';

  // 차트
  renderNDCChart(emissionsTS, target, gapInfo);

  // NDC 3.0 정보
  const ndc3El = document.getElementById('ndc3-info');
  if (target.ndc3) {
    ndc3El.style.display = 'block';
    ndc3El.innerHTML = `
      <strong>NDC 3.0 Submitted</strong> (${target.ndc3_date || ''})<br>
      ${target.ndc3_pct ? `2035 target: -${target.ndc3_pct}% from ${target.ref_year || 'ref year'}` :
'Details pending'}
    `;
  } else {
    ndc3El.style.display = 'block';
    ndc3El.innerHTML = `<strong>NDC 3.0:</strong> Not yet submitted as of April 2026`;
  }

  // Detail Table
  const detailEl = document.getElementById('ndc-detail');
  detailEl.innerHTML = `
    <tr><td>Reference type</td><td style="text-align:right;">${target.ref_type}</td></tr>
    ${target.ref_year ? `<tr><td>Reference year</td><td
style="text-align:right;">${target.ref_year}</td></tr>` : ''}
    ${target.ref_mt ? `<tr><td>Reference emissions</td><td
style="text-align:right;">${VC.fmt(target.ref_mt)} MtCO₂eq</td></tr>` : ''}
    ${target.target_mt ? `<tr><td>2030 target emissions</td><td
style="text-align:right;">${VC.fmt(target.target_mt)} MtCO₂eq</td></tr>` : ''}
    <tr><td>Net-zero legal status</td><td style="text-align:right;">${target.nz_legal.replace(/_/g,
' ')}</td></tr>
    ${gapInfo.latestEmissions ? `<tr><td>Latest emissions (${gapInfo.latestYear})</td><td
style="text-align:right;">${VC.fmt(gapInfo.latestEmissions)} MtCO₂eq</td></tr>` : ''}
    ${gapInfo.projected2030 != null ? `<tr><td>Projected 2030 (linear)</td><td
style="text-align:right;">${VC.fmt(gapInfo.projected2030)} MtCO₂eq</td></tr>` : ''}
```

```
    ${gapInfo.gapMt != null ? `<tr><td style="font-weight:600;">Gap (projected − target)</td><td
style="text-align:right;font-weight:600;color:${VC.GAP_COLOR[gapInfo.severity] ||
'#000'}">${gapInfo.gapMt > 0 ? '+' : ''}${VC.fmt(gapInfo.gapMt)} MtCO₂eq</td></tr>` : ''}
  `;
}

// ─── Gap 계산 (CF04/CF05 로직) ───
function calculateGap(ts, target) {
  const result = {
    severity: 'No Data',
    latestEmissions: null,
    latestYear: null,
    projected2030: null,
    gapMt: null,
    trendCAGR: null,
  };

  if (ts.length < 3) return result;

  // 최근 데이터
  const sorted = [...ts].sort((a, b) => b.year - a.year);
  const latest = sorted[0];
  result.latestEmissions = latest.value;
  result.latestYear = latest.year;

  // 10년 추세로 2030 투영 (CAGR)
  const tenYearAgo = sorted.find(r => r.year <= latest.year - 10);
  if (tenYearAgo && tenYearAgo.value > 0) {
    const years = latest.year - tenYearAgo.year;
    const cagr = Math.pow(latest.value / tenYearAgo.value, 1 / years) - 1;
    result.trendCAGR = cagr;
    result.projected2030 = latest.value * Math.pow(1 + cagr, 2030 - latest.year);
  } else {
    // fallback: 선형 추세
    const fiveYearAgo = sorted.find(r => r.year <= latest.year - 5);
    if (fiveYearAgo) {
      const slope = (latest.value - fiveYearAgo.value) / (latest.year - fiveYearAgo.year);
      result.projected2030 = latest.value + slope * (2030 - latest.year);
    }
  }

  // absolute 타입만 직접 갭 계산 가능
  if (target.ref_type === 'absolute' && target.target_mt && result.projected2030 != null) {
    result.gapMt = result.projected2030 - target.target_mt;
```

```
      // CF05 로직
      if (result.gapMt <= 0) {
        result.severity = 'On Track';
      } else if (result.gapMt / target.target_mt < 0.1) {
        result.severity = 'Narrow Gap';
      } else if (result.gapMt / target.target_mt < 0.3) {
        result.severity = 'Significant Gap';
      } else {
        result.severity = 'Critical Gap';
      }
    } else if (target.ref_type === 'intensity' || target.ref_type === 'bau') {
      // intensity/BAU 타입은 직접 비교 불가 → 추세만 표시
      if (result.trendCAGR != null) {
        result.severity = result.trendCAGR <= -0.02 ? 'On Track' :
                          result.trendCAGR <= 0 ? 'Narrow Gap' :
                          result.trendCAGR <= 0.02 ? 'Significant Gap' : 'Critical Gap';
      }
    }

    return result;
  }

  // ─── NDC 차트 (배출 궤적 + 목표선) ───
  function renderNDCChart(ts, target, gapInfo) {
    const container = document.getElementById('ndc-chart');
    if (ts.length === 0) {
      container.innerHTML = '<div class="empty-state"><p>No emissions data
available.</p></div>';
      return;
    }

    const sorted = [...ts].sort((a, b) => a.year - b.year);
    // 2000년 이후만 표시
    const recent = sorted.filter(r => r.year >= 2000);
    if (recent.length === 0) {
      container.innerHTML = '<div class="empty-state"><p>No post-2000 data.</p></div>';
      return;
    }

    const W = container.clientWidth || 500;
    const H = 260;
    const PAD = { top: 15, right: 50, bottom: 35, left: 55 };
    const plotW = W - PAD.left - PAD.right;
```

```
  const plotH = H - PAD.top - PAD.bottom;

  // 범위: x = 2000~2035, y = 자동
  const minYear = 2000;
  const maxYear = 2035;
  const values = recent.map(r => r.value);
  const extras = [];
  if (target.target_mt) extras.push(target.target_mt);
  if (target.ref_mt) extras.push(target.ref_mt);
  if (gapInfo.projected2030) extras.push(gapInfo.projected2030);
  const allVals = [...values, ...extras].filter(v => v != null);
  const minVal = Math.min(0, Math.min(...allVals)) * 0.9;
  const maxVal = Math.max(...allVals) * 1.15;

  const xScale = (year) => PAD.left + ((year - minYear) / (maxYear - minYear)) * plotW;
  const yScale = (val) => PAD.top + plotH - ((val - minVal) / (maxVal - minVal)) * plotH;

  let svg = `<svg width="${W}" height="${H}" viewBox="0 0 ${W} ${H}"
xmlns="http://www.w3.org/2000/svg">`;

  // 그리드
  for (let i = 0; i <= 4; i++) {
    const v = minVal + (maxVal - minVal) * (i / 4);
    const y = yScale(v);
    svg += `<line x1="${PAD.left}" y1="${y}" x2="${W - PAD.right}" y2="${y}" stroke="#E5E7EB"
stroke-width="0.5"/>`;
    svg += `<text x="${PAD.left - 4}" y="${y + 3}" text-anchor="end" font-size="8" fill="#8888A0"
font-family="JetBrains Mono,monospace">${formatVal(v)}</text>`;
  }

  // X축
  for (let y = 2000; y <= 2035; y += 5) {
    const x = xScale(y);
    svg += `<line x1="${x}" y1="${PAD.top}" x2="${x}" y2="${PAD.top + plotH}"
stroke="#E5E7EB" stroke-width="0.3"/>`;
    svg += `<text x="${x}" y="${H - 10}" text-anchor="middle" font-size="8" fill="#8888A0"
font-family="JetBrains Mono,monospace">${y}</text>`;
  }

  // 2030 목표선 (absolute 타입)
  if (target.target_mt && target.ref_type === 'absolute') {
    const ty = yScale(target.target_mt);
    svg += `<line x1="${PAD.left}" y1="${ty}" x2="${W - PAD.right}" y2="${ty}" stroke="#E5484D"
stroke-width="1.5" stroke-dasharray="6,3"/>`;
```

```
    svg += `<text x="${W - PAD.right + 3}" y="${ty + 3}" font-size="8" fill="#E5484D"
font-family="Inter,sans-serif" font-weight="600">Target ${VC.fmt(target.target_mt, 0)}</text>`;
  }

  // 갭 영역 (projected와 target 사이)
  if (gapInfo.projected2030 != null && target.target_mt && target.ref_type === 'absolute') {
    const latestX = xScale(gapInfo.latestYear);
    const projX = xScale(2030);
    const projY = yScale(gapInfo.projected2030);
    const targetY = yScale(target.target_mt);
    const gapColor = gapInfo.gapMt > 0 ? 'rgba(229,72,77,0.12)' : 'rgba(0,166,126,0.12)';
    svg += `<polygon points="${latestX},${yScale(gapInfo.latestEmissions)} ${projX},${projY}
${projX},${targetY} ${latestX},${targetY}" fill="${gapColor}"/>`;
  }

  // 실배출 라인 (실선)
  const pathParts = recent.map((r, i) => {
    const x = xScale(r.year);
    const y = yScale(r.value);
    return i === 0 ? `M ${x} ${y}` : `L ${x} ${y}`;
  });
  svg += `<path d="${pathParts.join(' ')}" fill="none" stroke="#0066FF" stroke-width="2"/>`;

  // 투영 점선 (latest → 2030)
  if (gapInfo.projected2030 != null && gapInfo.latestEmissions != null) {
    const lx = xScale(gapInfo.latestYear);
    const ly = yScale(gapInfo.latestEmissions);
    const px = xScale(2030);
    const py = yScale(gapInfo.projected2030);
    svg += `<line x1="${lx}" y1="${ly}" x2="${px}" y2="${py}" stroke="#0066FF"
stroke-width="1.5" stroke-dasharray="4,3" opacity="0.6"/>`;
    svg += `<circle cx="${px}" cy="${py}" r="4" fill="#0066FF" opacity="0.6"/>`;
    svg += `<text x="${px + 5}" y="${py + 3}" font-size="8" fill="#0066FF" font-family="JetBrains
Mono,monospace">${VC.fmt(gapInfo.projected2030, 0)}</text>`;
  }

  // 최신 데이터 포인트
  if (recent.length > 0) {
    const last = recent[recent.length - 1];
    svg += `<circle cx="${xScale(last.year)}" cy="${yScale(last.value)}" r="4" fill="#0066FF"/>`;
  }

  svg += '</svg>';
  container.innerHTML = svg;
```

```
  }

  function formatVal(v) {
    if (Math.abs(v) >= 1e3) return (v / 1e3).toFixed(0) + 'Gt';
    return v.toFixed(0) + 'Mt';
  }

  function showNotCovered(iso3) {
    hideAll();
    const el = document.getElementById('empty-state');
    el.style.display = 'flex';
    el.innerHTML = `<p>${iso3} is not in the 20-country NDC target list.</p>
      <p style="font-size:11px;color:#8888A0;margin-top:8px;">
        Covered countries: ${Object.keys(VC.NDC_TARGETS).join(', ')}
      </p>`;
  }

  function hideAll() {
    ['setup-screen', 'loading', 'empty-state', 'ndc-main'].forEach(function (id) {
      document.getElementById(id).style.display = 'none';
    });
  }

  function showEmptyState() {
    hideAll();
    document.getElementById('empty-state').style.display = 'flex';
  }
})();
```

배포 및 사용 가이드

로컬 테스트

```
# 1. extension/ 폴더를 visualclimate-trojan 레포에 추가
cd visualclimate-trojan
mkdir extension
# (위의 모든 파일을 extension/ 에 저장)

# 2. 간단한 HTTP 서버 시작
cd extension
npx http-server -p 8765 --cors
# 또는: python3 -m http.server 8765
```

```
# 3. Tableau Desktop 열기
# Dashboard → Objects → Extension → Access Local Extensions
# extension/visualclimate.trex 선택
# 최초 실행 시 Supabase URL과 Key 입력 화면 표시됨
```

Vercel 배포 (Production)

```
# extension/ 폴더를 별도 레포 또는 서브디렉토리로 배포
cd extension
vercel --prod

# 배포 후 .trex 파일의 <url> 수정:
# <url>https://ext.visualclimate.org/index.html</url>
# <url>https://ext.visualclimate.org/timeseries.html</url>
# <url>https://ext.visualclimate.org/ndc-tracker.html</url>

# Tableau Cloud/Server에서 사용 시 Allowlist 등록 필요
```

대시보드 레이아웃 (tableau-setup.md 기반)

```
┌──────────────────────────────────────────────────────────────┐
│ [Filter: Country Dropdown (ISO3)]                            │
├──────────────────────────┬───────────────────────────────────┤
│                          │                                   │
│   World Map              │   Climate Card Extension          │
│   (Filled Map)           │   (index.html)                    │
│   1200×400               │   480×800                         │
│                          │                                   │
├──────────────────────────┤                                   │
│                          │                                   │
│   Emissions Line         │                                   │
│   (timeseries ws)        │                                   │
│   or Timeseries          │                                   │
│   Extension              │                                   │
│   (timeseries.html)      │                                   │
│                          │                                   │
└──────────────────────────┴───────────────────────────────────┘
```

3개 Extension 요약

Extension	파일	데이터 소스	기능

Climate Card	`index.html` + `app.js`	Supabase `REPORT.*` + `DERIVED.CLIMATE_CLASS`	스코어카드, 등급배지, 5도메인 바/레이더, 피어랭킹, 인사이트
Timeseries Explorer	`timeseries.html` + `timeseries-app.js`	Supabase `country_data` (7개 그룹, 30+ indicator_code)	SVG 라인차트, 지표 드롭다운, 그룹 전환, 데이터 테이블
NDC Gap Tracker	`ndc-tracker.html` + `ndc-tracker-app.js`	`ndc-targets-v2.json` (20국) + Supabase 실배출 시계열	목표선, 투영선, 갭영역, 심각도 배지, CAT 등급, NZ 카운트다운

모든 코드는 실제 Supabase 테이블 구조(`countries.iso3`, `country_data.country_iso3`, `country_data.indicator_code`), 실제 indicator_code 패턴(`REPORT.*`, `OWID.*`, `CTRACE.*`, `EMBER.*`, `NDGAIN.*`, `DERIVED.*`, WB 코드), `brand-config.md` 색상, `ndc-targets-v2.json` 20국 데이터, `ReportCardClient.tsx`의 GRADE_LABEL/CLASS_PILL/DOMAINS 구조를 정확히 반영합니다.